Szyfr afiniczny

Marcin Kocot

Zadanie wykonano w języku Kotlin, przy użyciu build-toola Bazel.

Funkcje pomocnicze

Największy wspólny dzielnik

```
tailrec fun gcd(a: Int, b: Int): Int {
  if (b == 0) {
    return a
  }
  return gcd(b, a % b)
}
```

Odwrotność modulo

```
        private infix fun Int.invMod(mod: Int): Int {
          var a = this
          var m0 = mod
          var y = 0
          var x = 1

          if (mod == 1) return 0

          while (a > 1) {
            val q = a / m0
            var t = m0

            m0 = a % m0
            a = t
            t = y

            y = x - q * y
            x = t
          }

          if (x < 0) x += mod

          return x
        }
```

Wyznaczanie najczęściej występujących w zbiorze liter

```
        private fun Map<Char, Int>.getTopN(n: Int): List<Char>
=
          this.entries
            .sortedByDescending { it.value }
            .take(n)
            .map { it.key }
```

Wyznaczanie wszystkich możliwych par liter

```
        private fun List<Char>.getAllPairs(): List<Pair<Char,
Char>> {
          val res = mutableListOf<Pair<Char, Char>>()
          for (a in this) {
            for (b in this) {
              if (a != b) res.add(a to b)
            }
          }
          return res
        }
```

Inne

```
        // Wykonanie operacji na każdej literze napisu
        private fun String.mapLetters(operation: (Char) ->
Char): String =
          this.uppercase().map{
            if (it.isLetter()) operation(it) else it
          }.joinToString(separator = "")

        // Zwraca kod ASCII litery
        private fun Char.ord(): Int =
          this.code - 'A'.code

        // Zwraca literę z danym kodem ASCII
        private fun Int.chr(): Char =
          Char(this + 'A'.code)

        // Wylicza funkcję ax+b modulo 26
        private fun Int.ab(a: Int, b: Int): Int =
          (a * this + b) posMod  26

        // Wylicza odwrotność funkcji ax+b modulo 26
        private fun Int.revAb(a: Int, b: Int): Int =
          ((this - b) * (a invMod 26)) posMod 26

        // Wylicza nieujemną resztę z dzielenia
        private infix fun Int.posMod(div: Int): Int =
          (this % div).let { if (it >= 0) it else it + div }
```

Szyfrowanie

```
        private fun String.encrypt(a: Int, b: Int): String =
          this.mapLetters { it.ord().ab(a, b).chr() }

        private fun String.decrypt(a: Int, b: Int): String =
          this.mapLetters { it.ord().revAb(a, b).chr() }
```

Testowanie

Weryfikacja poprawności klucza

```
        private fun verifyAB(a: Int, b: Int): Boolean =
          a > 0 && b >= 0 && a < 26 && gcd(a, 26) == 1 && b <
26
```

Wyznaczanie klucza na podstawie par liter

```
        private fun solveWithPairs(p1: Pair<Char, Char>, p2:
Pair<Char, Char>): Pair<Int, Int>? {
          val a1 = p1.first.ord()
          val a2 = p2.first.ord()
          val b1 = p1.second.ord()
          val b2 = p2.second.ord()
          var diffA = a1 - a2
          var diffB = b1 - b2
          if (diffA < 0) {
            diffA = -diffA
            diffB = -diffB
          }
          if ((diffA % 2 == 0) != (diffB % 2 == 0)) return null
// if they have different parities, adding 26 will not help
          while (diffB % diffA != 0) diffB += 26
          val solutionA = diffB / diffA
          val solutionB = (b1 - a1 * solutionA) posMod 26
          return solutionA to solutionB
        }
```

Próby wyznaczania kluczy na podstawie N najczęściej występujących liter

```
        private fun testNxN(cypher: String, n: Int):
List<String> {
          val results = mutableListOf<String>()
          val upc = cypher.uppercase()
          val counts = PROBS.keys.associateWith { letter ->
upc.count { it == letter } }
          for ((a1, a2) in PROBS.getTopN(n).getAllPairs()) {
            for ((b1, b2) in counts.getTopN(n).getAllPairs()) {
              val sol = solveWithPairs(a1 to b1, a2 to b2) ?:
continue
              val body = if (verifyAB(sol.first, sol.second)) {
                cypher.decrypt(sol.first, sol.second).also {
results.add(it) }
              } else "<Incorrect key>"
            }
          }
          return results
        }
```

Pełne testowanie

To jest najważniejsza metoda całego programu - wyznacza klucze za pomocą dwóch najczęściej występujących liter w zaszyfrowanej wiadomości i ogólnie w języku angielskim. Jeśli się nie powiedzie, ponawiamy dla trzech najczęstszych liter, potem czterech itd.

Dla bardzo długich testów, pierwsze wykonanie wystarczało. Im krótszy tekst, tym więcej prób trzeba było podjąć.

```
        private fun testUntilSuccessful(cypher: String,
goalPlain: String, maxN: Int): Int {
          val goalUpper = goalPlain.uppercase()
          repeat(maxN - 1) {
            val n = it + 2
            val results = testNxN(cypher, n)
            if (results.contains(goalUpper)) return n
          }
          return 999
        }
```